

Sotility Ecosystem Enhancement: Recommended Smart Contracts & Architectural Improvements

After analyzing the current Sotility smart contract architecture and researching the latest innovations in DeFi, tokenomics, and on-chain reputation systems, I recommend the following enhancements to create a more robust, secure, and innovative economic system.

1. Security and Risk Mitigation Improvements

1.1. SotilityEmergencyShutdown.sol

Plain Text

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
import "@openzeppelin/contracts/access/AccessControl.sol";

/**
 * @title SotilityEmergencyShutdown
 * @dev Circuit breaker pattern for Sotility ecosystem in case of attacks
 */
contract SotilityEmergencyShutdown is AccessControl {
    bytes32 public constant GUARDIAN_ROLE = keccak256("GUARDIAN_ROLE");
    bytes32 public constant RECOVERY_ROLE = keccak256("RECOVERY_ROLE");

    mapping(address => bool) public pausedContracts;
    bool public systemWideEmergency;
    uint256 public emergencyTimelock;
    uint256 public constant EMERGENCY_DELAY = 24 hours;

    event ContractPaused(address indexed contractAddress);
    event ContractUnpaused(address indexed contractAddress);
    event SystemWideEmergencyActivated();
    event SystemWideEmergencyDeactivated();
    event EmergencyTimelockInitiated(uint256 activationTime);

    constructor(address[] memory guardians, address[] memory recoveryTeam) {
        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);

        for (uint i = 0; i < guardians.length; i++) {
            _grantRole(GUARDIAN_ROLE, guardians[i]);
        }
    }
}
```

```

        for (uint i = 0; i < recoveryTeam.length; i++) {
            _grantRole(RECOVERY_ROLE, recoveryTeam[i]);
        }
    }

    function initiateEmergencyTimelock() external onlyRole(GUARDIAN_ROLE) {
        require(emergencyTimelock == 0, "Timelock already initiated");
        emergencyTimelock = block.timestamp + EMERGENCY_DELAY;
        emit EmergencyTimelockInitiated(emergencyTimelock);
    }

    function activateSystemWideEmergency() external onlyRole(GUARDIAN_ROLE) {
        require(block.timestamp >= emergencyTimelock, "Emergency timelock
not expired");
        systemWideEmergency = true;
        emit SystemWideEmergencyActivated();
    }

    function deactivateSystemWideEmergency() external
onlyRole(RECOVERY_ROLE) {
        systemWideEmergency = false;
        emergencyTimelock = 0;
        emit SystemWideEmergencyDeactivated();
    }

    function pauseContract(address contractAddress) external
onlyRole(GUARDIAN_ROLE) {
        pausedContracts[contractAddress] = true;
        emit ContractPaused(contractAddress);
    }

    function unpauseContract(address contractAddress) external
onlyRole(RECOVERY_ROLE) {
        pausedContracts[contractAddress] = false;
        emit ContractUnpaused(contractAddress);
    }

    function isContractPaused(address contractAddress) external view returns
(bool) {
        return pausedContracts[contractAddress] || systemWideEmergency;
    }
}

```

1.2. MultiOracleAggregator.sol

Plain Text

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
import "@openzeppelin/contracts/access/AccessControl.sol";

/**
 * @title MultiOracleAggregator
 * @dev Aggregates data from multiple AI oracles to prevent oracle
manipulation
 */
contract MultiOracleAggregator is AccessControl {
    bytes32 public constant ORACLE_ADMIN_ROLE =
keccak256("ORACLE_ADMIN_ROLE");

    struct OracleData {
        uint256 score;
        uint256 timestamp;
        bool submitted;
    }

    struct ContentScore {
        mapping(address => OracleData) oracleScores;
        address[] oracles;
        uint256 finalScore;
        bool finalized;
    }

    mapping(uint256 => ContentScore) public contentScores;
    mapping(address => bool) public authorizedOracles;
    uint256 public minimumOracleResponses;
    uint256 public scoreDeviationThreshold; // in basis points (e.g., 1000 =
10%)

    event OracleAuthorized(address indexed oracle);
    event OracleDeauthorized(address indexed oracle);
    event ScoreSubmitted(uint256 indexed contentId, address indexed oracle,
uint256 score);
    event ScoreFinalized(uint256 indexed contentId, uint256 finalScore);

    constructor(uint256 _minimumOracleResponses, uint256
_scoreDeviationThreshold) {
        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
        _grantRole(ORACLE_ADMIN_ROLE, msg.sender);
        minimumOracleResponses = _minimumOracleResponses;
        scoreDeviationThreshold = _scoreDeviationThreshold;
    }

    function authorizeOracle(address oracle) external

```

```

onlyRole(ORACLE_ADMIN_ROLE) {
    authorizedOracles[oracle] = true;
    emit OracleAuthorized(oracle);
}

function deauthorizeOracle(address oracle) external
onlyRole(ORACLE_ADMIN_ROLE) {
    authorizedOracles[oracle] = false;
    emit OracleDeauthorized(oracle);
}

function submitScore(uint256 contentId, uint256 score) external {
    require(authorizedOracles[msg.sender], "Not an authorized oracle");
    require(score <= 100, "Score must be between 0-100");
    require(!contentScores[contentId].finalized, "Score already
finalized");

    require(!contentScores[contentId].oracleScores[msg.sender].submitted,
"Oracle already submitted");

    ContentScore storage content = contentScores[contentId];
    content.oracleScores[msg.sender] = OracleData({
        score: score,
        timestamp: block.timestamp,
        submitted: true
    });
    content.oracles.push(msg.sender);

    emit ScoreSubmitted(contentId, msg.sender, score);

    if (content.oracles.length >= minimumOracleResponses) {
        _attemptFinalization(contentId);
    }
}

function _attemptFinalization(uint256 contentId) internal {
    ContentScore storage content = contentScores[contentId];

    // Calculate average score
    uint256 totalScore;
    for (uint i = 0; i < content.oracles.length; i++) {
        totalScore += content.oracleScores[content.oracles[i]].score;
    }
    uint256 averageScore = totalScore / content.oracles.length;

    // Check for large deviations
    bool hasLargeDeviation = false;
    for (uint i = 0; i < content.oracles.length; i++) {

```

```

        uint256 oracleScore =
content.oracleScores[content.oracles[i]].score;
        uint256 deviation;

        if (oracleScore > averageScore) {
            deviation = ((oracleScore - averageScore) * 10000) /
averageScore;
        } else {
            deviation = ((averageScore - oracleScore) * 10000) /
averageScore;
        }

        if (deviation > scoreDeviationThreshold) {
            hasLargeDeviation = true;
            break;
        }
    }

    if (!hasLargeDeviation) {
        content.finalScore = averageScore;
        content.finalized = true;
        emit ScoreFinalized(contentId, averageScore);
    }
}

function getScoreStatus(uint256 contentId) external view returns (
    uint256 oracleCount,
    bool isFinalized,
    uint256 finalScore
) {
    ContentScore storage content = contentScores[contentId];
    return (
        content.oracles.length,
        content.finalized,
        content.finalScore
    );
}

function forceFinalize(uint256 contentId) external
onlyRole(ORACLE_ADMIN_ROLE) {
    ContentScore storage content = contentScores[contentId];
    require(!content.finalized, "Already finalized");
    require(content.oracles.length >= minimumOracleResponses, "Not
enough oracles");

    uint256 totalScore;
    for (uint i = 0; i < content.oracles.length; i++) {
        totalScore += content.oracleScores[content.oracles[i]].score;
    }
}

```

```

    }
    uint256 averageScore = totalScore / content.oracles.length;

    content.finalScore = averageScore;
    content.finalized = true;

    emit ScoreFinalized(contentId, averageScore);
  }
}

```

2. Token Mechanism Improvements

2.1. SotilityVeToken.sol

Plain Text

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
import "@openzeppelin/contracts/token/ERC20/extensions/ERC20Burnable.sol";
import "@openzeppelin/contracts/access/AccessControl.sol";

/**
 * @title SotilityVeToken
 * @dev Vote-escrowed token model for longer-term governance commitment
 */
contract SotilityVeToken is AccessControl {
    bytes32 public constant TOKEN_MANAGER_ROLE =
    keccak256("TOKEN_MANAGER_ROLE");

    struct LockPosition {
        uint256 amount;
        uint256 lockEnd;
        uint256 veAmount;
    }

    // Token being locked
    address public immutable baseToken;

    // Maximum lock time (4 years)
    uint256 public constant MAX_LOCK_TIME = 4 * 365 days;

    // User balances
    mapping(address => LockPosition) public lockPositions;

    // Total veToken supply
    uint256 public totalVeSupply;

```

```

    event TokensLocked(address indexed user, uint256 amount, uint256
lockEnd, uint256 veAmount);
    event LockIncreased(address indexed user, uint256 newAmount, uint256
newLockEnd, uint256 newVeAmount);
    event TokensUnlocked(address indexed user, uint256 amount);

    constructor(address _baseToken) {
        baseToken = _baseToken;
        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
        _grantRole(TOKEN_MANAGER_ROLE, msg.sender);
    }

    /**
     * @dev Lock tokens for a period of time to receive veTokens
     * @param amount Amount of base tokens to lock
     * @param lockDuration Duration to lock tokens in seconds (up to
MAX_LOCK_TIME)
     */
    function lockTokens(uint256 amount, uint256 lockDuration) external {
        require(amount > 0, "Amount must be greater than 0");
        require(lockDuration > 0 && lockDuration <= MAX_LOCK_TIME, "Invalid
lock duration");

        IERC20(baseToken).transferFrom(msg.sender, address(this), amount);

        uint256 lockEnd = block.timestamp + lockDuration;
        uint256 veAmount = calculateVeAmount(amount, lockDuration);

        LockPosition storage position = lockPositions[msg.sender];

        if (position.amount > 0) {
            // User already has a lock position, merge them
            position.amount += amount;
            position.lockEnd = lockEnd > position.lockEnd ? lockEnd :
position.lockEnd;

            // Recalculate veAmount based on new values
            uint256 remainingDuration = position.lockEnd - block.timestamp;
            position.veAmount = calculateVeAmount(position.amount,
remainingDuration);
        } else {
            // New lock position
            position.amount = amount;
            position.lockEnd = lockEnd;
            position.veAmount = veAmount;
        }
    }

```

```

        totalVeSupply += veAmount;

        emit TokensLocked(msg.sender, amount, lockEnd, veAmount);
    }

    /**
     * @dev Extend lock duration to get more veTokens
     * @param newDuration New lock duration in seconds from now
     */
    function extendLock(uint256 newDuration) external {
        LockPosition storage position = lockPositions[msg.sender];
        require(position.amount > 0, "No existing lock");
        require(position.lockEnd > block.timestamp, "Lock expired");
        require(newDuration > 0, "Duration must be greater than 0");

        uint256 newLockEnd = block.timestamp + newDuration;
        require(newLockEnd > position.lockEnd, "New lock end must be later
than current");
        require(newLockEnd <= block.timestamp + MAX_LOCK_TIME, "Lock too
long");

        // Remove old veTokens from total supply
        totalVeSupply -= position.veAmount;

        // Update lock position
        position.lockEnd = newLockEnd;
        position.veAmount = calculateVeAmount(position.amount, newDuration);

        // Add new veTokens to total supply
        totalVeSupply += position.veAmount;

        emit LockIncreased(msg.sender, position.amount, newLockEnd,
position.veAmount);
    }

    /**
     * @dev Unlock tokens after lock period ends
     */
    function unlock() external {
        LockPosition storage position = lockPositions[msg.sender];
        require(position.amount > 0, "No existing lock");
        require(block.timestamp >= position.lockEnd, "Lock not expired yet");

        uint256 amount = position.amount;

        // Remove veTokens from total supply
        totalVeSupply -= position.veAmount;
    }

```

```

        // Clear lock position
        position.amount = 0;
        position.lockEnd = 0;
        position.veAmount = 0;

        // Return tokens to user
        IERC20(baseToken).transfer(msg.sender, amount);

        emit TokensUnlocked(msg.sender, amount);
    }

    /**
     * @dev Calculate veToken amount based on lock duration
     * @param amount Base token amount
     * @param duration Lock duration in seconds
     */
    function calculateVeAmount(uint256 amount, uint256 duration) public pure
    returns (uint256) {
        // Linear scaling: 1 year = 0.25x, 2 years = 0.5x, 3 years = 0.75x,
        4 years = 1x
        return (amount * duration) / MAX_LOCK_TIME;
    }

    /**
     * @dev Get current veToken balance for an account
     * @param account Address to check
     */
    function balanceOf(address account) external view returns (uint256) {
        LockPosition storage position = lockPositions[account];

        if (position.lockEnd <= block.timestamp) {
            return 0; // Lock expired, no voting power
        }

        return position.veAmount;
    }

    /**
     * @dev Get detailed lock position info
     */
    function getLockPosition(address account) external view returns (
        uint256 amount,
        uint256 lockEnd,
        uint256 veAmount,
        uint256 remainingTime
    ) {
        LockPosition storage position = lockPositions[account];
        uint256 remaining = position.lockEnd > block.timestamp ?

```

```

position.lockEnd - block.timestamp : 0;

        return (
            position.amount,
            position.lockEnd,
            position.veAmount,
            remaining
        );
    }
}

```

2.2. SotilityLiquidStaking.sol

Plain Text

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
import "@openzeppelin/contracts/token/ERC20/extensions/ERC20Burnable.sol";
import "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import "@openzeppelin/contracts/access/AccessControl.sol";

/**
 * @title SotilityLiquidStaking
 * @dev Allows users to stake SOT while receiving liquid staked tokens
 (lstSOT)
 */
contract SotilityLiquidStaking is ERC20, AccessControl {
    bytes32 public constant VALIDATOR_ROLE = keccak256("VALIDATOR_ROLE");
    bytes32 public constant FEE_COLLECTOR_ROLE =
keccak256("FEE_COLLECTOR_ROLE");

    address public immutable stakingToken; // SOT

    uint256 public protocolFee; // Basis points (100 = 1%)
    uint256 public totalStaked;
    uint256 public totalRewards;
    uint256 public constant FEE_DENOMINATOR = 10000;

    event Staked(address indexed user, uint256 amount, uint256 lstAmount);
    event Unstaked(address indexed user, uint256 lstAmount, uint256
sotAmount);
    event RewardsAdded(uint256 amount);
    event ProtocolFeeUpdated(uint256 newFee);

    constructor(address _stakingToken, uint256 _initialFee) ERC20("Liquid
Staked SOT", "lstSOT") {
        stakingToken = _stakingToken;
    }
}

```

```

        protocolFee = _initialFee;
        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
        _grantRole(VALIDATOR_ROLE, msg.sender);
        _grantRole(FEE_COLLECTOR_ROLE, msg.sender);
    }

    /**
     * @dev Stake SOT tokens and receive lstSOT in return
     * @param amount Amount of SOT to stake
     */
    function stake(uint256 amount) external {
        require(amount > 0, "Cannot stake 0");

        // Transfer SOT from user
        IERC20(stakingToken).transferFrom(msg.sender, address(this), amount);

        // Calculate lstSOT to mint based on current exchange rate
        uint256 lstAmount = (amount * 1e18) / exchangeRate();

        // Mint lstSOT to user
        _mint(msg.sender, lstAmount);

        // Update total staked
        totalStaked += amount;

        emit Staked(msg.sender, amount, lstAmount);
    }

    /**
     * @dev Unstake SOT by burning lstSOT
     * @param lstAmount Amount of lstSOT to burn
     */
    function unstake(uint256 lstAmount) external {
        require(lstAmount > 0, "Cannot unstake 0");
        require(balanceOf(msg.sender) >= lstAmount, "Insufficient lstSOT
balance");

        // Calculate SOT amount based on current exchange rate
        uint256 sotAmount = (lstAmount * exchangeRate()) / 1e18;

        // Burn lstSOT
        _burn(msg.sender, lstAmount);

        // Calculate protocol fee
        uint256 feeAmount = (sotAmount * protocolFee) / FEE_DENOMINATOR;
        uint256 sotToReturn = sotAmount - feeAmount;

```

// Transfer